

Máster en Inteligencia Artificial
para Liderazgo Técnico y Transformación Empresarial

CI/CD para Sistemas de Inteligencia Artificial

Módulo 8 · Ingeniería de Modelos y LLMOps

Pipeline CI/CD para LLM, canary deployment, feature flags, regression tests y el ROI de la automatización.

1. Introducción · 2. Qué es · 3. Cómo funciona · 4. Ejemplos reales
5. Errores comunes · 6. Aplicación práctica · 7. Relación con módulos · 8. Resumen ejecutivo

Guía de estudio detallada · +2.000 palabras

CI/CD para Sistemas de Inteligencia Artificial

CI/CD (Continuous Integration / Continuous Delivery) es la práctica de software que automatiza la validación y el despliegue de cambios al sistema. En sistemas de IA, CI/CD enfrenta desafíos únicos que no existen en software tradicional: los tests no son deterministas (el mismo modelo con temperatura >0 produce outputs distintos), la calidad no se puede verificar con una comparación exacta sino con métricas semánticas, y el "código" que hay que validar incluye prompts y datos además del código Python.

A pesar de estos desafíos, CI/CD para sistemas de IA es fundamental para operar a escala. Sin un pipeline automatizado que valide cada cambio antes del despliegue, el proceso de mejora es lento (evaluación manual en lugar de automatizada), propenso a errores (cambios que degradan la calidad llegan a producción), y no escalable (la carga de evaluación manual crece linealmente con la velocidad de desarrollo).

Los componentes del CI/CD para IA

Continuous Integration: validar antes de mergear

La CI de un sistema LLM valida automáticamente cada Pull Request con: linting y tests unitarios del código Python, validación del formato del prompt (si el schema del prompt se respeta), evaluación del golden dataset con las métricas definidas, comparación de métricas con la versión actual del sistema (si las métricas bajan más de un umbral definido, el PR es bloqueado automáticamente), y un resumen automático de las métricas en el PR para facilitar la revisión.

Continuous Delivery: despliegue gradual y controlado

La CD de un sistema LLM implementa despliegue gradual para minimizar el riesgo: shadow mode (el nuevo sistema procesa las mismas consultas que el sistema actual pero sin devolver las respuestas al usuario, para medir calidad sin riesgo), canary deployment (el nuevo sistema recibe un porcentaje pequeño del tráfico real, típicamente 5-10%, mientras se monitoriza la calidad en comparación con el sistema actual), y rollout completo (solo cuando las métricas del canary son iguales o mejores que el sistema actual).

Regression testing: el corazón del CI para LLM

Los regression tests verifican que los cambios no degradan el rendimiento en los casos que el sistema ya resolvía correctamente. Para sistemas LLM, los regression tests se ejecutan automáticamente sobre el golden dataset en cada CI. Los tipos de regression tests son: tests de calidad (métricas sobre el golden dataset), tests de seguridad (el sistema no produce outputs peligrosos en un test set de red team), tests de latencia (la latencia P99 no supera el umbral definido), y tests de coste (el coste por consulta no supera el presupuesto definido).

Implementación de CI/CD para sistemas LLM

Pipeline de CI con GitHub Actions

Un pipeline de CI para un sistema LLM en GitHub Actions tiene los siguientes jobs: lint (ruff/black para Python), test-unit (pytest para tests unitarios del código no-LLM), evaluate-golden-dataset (ejecutar el sistema sobre el golden dataset con temperatura 0, calcular métricas, comparar con baseline), safety-tests (ejecutar un red team test set para verificar que el sistema no produce contenido dañino), y performance-test (medir latencia y coste por consulta con un sample del golden dataset). Si cualquier job falla o las métricas caen más del umbral, el PR es bloqueado.

Feature flags para control de despliegues

Los feature flags permiten activar o desactivar funcionalidades del sistema sin desplegar código. Para sistemas LLM, los feature flags más útiles son: model_version (qué versión del modelo usar), prompt_version (qué versión del prompt usar), rag_enabled (si RAG está activo), y features experimentales (nuevas capacidades que se prueban con un subconjunto de usuarios). Los feature flags permiten implementar canary deployments a nivel lógico sin necesidad de infraestructura de routing separada.

Automatización del ciclo de mejora

En sistemas LLM maduros, parte del ciclo de mejora se puede automatizar: detectar automáticamente los casos de peor rendimiento en producción (usando las métricas de monitorización), generar automáticamente variaciones del prompt usando un LLM, evaluar esas variaciones sobre el golden dataset en CI, y hacer una propuesta de PR con la mejor variante encontrada para revisión humana. Herramientas como DSPy y TextGrad automatizan la optimización de prompts en el contexto del pipeline de CI.

SECCIÓN 4 · EJEMPLOS REALES

- GitHub Actions CI para sistema RAG (empresa SaaS): el pipeline de CI ejecuta automáticamente RAGAS sobre 150 pares de Q&A; del golden dataset en cada PR. Si faithfulness baja más de 2pp o context recall baja más de 3pp respecto al baseline, el PR es bloqueado con un comentario explicando qué métrica falló y en qué ejemplos. El proceso completo toma 8 minutos. Antes de implementarlo, los cambios regresivos llegaban a producción regularmente.
- Canary deployment automático (startup fintech): el sistema de CD despliega automáticamente al 5% del tráfico cualquier cambio que pasa CI. Durante 24 horas, monitoriza el score LLM-as-judge, el CSAT del usuario, y la latencia del canary vs el sistema actual. Si todas las métricas son iguales o mejores, el rollout completo es automático. Si alguna métrica es peor, el canary es automáticamente revertido y el equipo recibe una alerta.
- Feature flags en sistema de chatbot (empresa de telecomunicaciones): lanzaron la integración RAG como feature flag activado solo para el 5% de los usuarios, monitorizando el CSAT y la tasa de resolución. Después de 2 semanas con datos positivos, lo activaron para el 50% y finalmente para el 100%. Si los resultados hubieran sido negativos, un toggle en el dashboard habría revertido al sistema anterior en segundos.

SECCIÓN 5 · ERRORES Y MALENTENDIDOS COMUNES

Error 1: Golden dataset demasiado pequeño para CI fiable. Con 20 ejemplos en el golden dataset, las métricas tienen alta varianza estadística: una diferencia de 2 ejemplos bien/mal resueltos produce una variación del 10% en la métrica. Para CI fiable, el golden dataset debe tener mínimo 50-100 casos para que las métricas sean estadísticamente significativas.

Error 2: Tests de CI demasiado lentos. Un pipeline de CI que tarda 30 minutos en ejecutarse frena el proceso de desarrollo y desencoraja su uso. Optimiza el CI LLM: usa el modelo más barato que produce métricas fiables (GPT-4o-mini en lugar de GPT-4 para el evaluador), paraleliza las evaluaciones, y ejecuta solo los tests más críticos en cada PR (el conjunto completo en nightly builds).

Error 3: No actualizar el golden dataset con el tiempo. El golden dataset construido el día 1 puede no representar los casos de uso actuales 6 meses después. Define un proceso para expandir el golden dataset regularmente con casos representativos del uso real del sistema en producción (seleccionados de los logs de monitorización).

SECCIÓN 6 · APLICACIÓN PRÁCTICA

El pipeline mínimo de CI/CD para un sistema LLM: CI (GitHub Actions/GitLab CI): lint del código, tests unitarios del código, evaluación sobre golden dataset con métricas definidas y comparación con baseline, test de seguridad con red team set básico. CD: despliegue automático al ambiente staging tras CI exitoso, canary 5% durante 24h con monitorización de métricas, rollout automático si métricas son OK o alerta y rollback si no. Feature flags para control granular de nuevas funcionalidades.

Para el step de evaluación en CI con Python: `import subprocess, result = subprocess.run(["python", "evaluate.py", "--dataset", "golden_dataset.json", "--model", model_version, "--prompt", prompt_version], capture_output=True)`. Parsea las métricas del output, compara con el baseline almacenado en un archivo JSON de referencia, y falla el CI si cualquier métrica cae más del umbral definido. El baseline se actualiza automáticamente cuando se hace un rollout exitoso.

SECCIÓN 7 · RELACIÓN CON OTROS MÓDULOS

- M8-T7 (Observabilidad): la observabilidad en producción proporciona los datos que retroalimentan el ciclo de CI/CD.
- M8-T8 (Versionado): el CI/CD se construye sobre el versionado de todos los componentes del sistema.
- M6-T8 (Evaluación de prompts): el golden dataset y las métricas de evaluación son el corazón del CI para sistemas LLM.

SECCIÓN 8 · RESUMEN EJECUTIVO

CI/CD para sistemas LLM valida automáticamente cada cambio antes del despliegue con: lint + tests unitarios, evaluación del golden dataset con comparación de métricas vs baseline (CI bloqueado si las métricas caen >umbral), red team tests de seguridad, y tests de latencia/coste. El despliegue es gradual: shadow mode → canary 5% (24h de

monitorización) → rollout completo si métricas OK. Feature flags para control granular. Los challenges únicos de LLM son: non-determinism (solución: temperatura 0 en CI o múltiples runs promediados) y métricas semánticas (solución: LLM-as-judge en el pipeline). ROI: 40-60% menos regresiones, 30-50% más velocidad de iteración.